

Agentes *no Cursor*

O **Agent** é o assistente do Cursor que conclui tarefas de programação por conta própria: lê e busca no projeto, edita vários arquivos, roda comandos no terminal e confere o resultado. Nesta aula: do que ele é feito, que ferramentas usa, os quatro modos, como acompanhar, corrigir o rumo e desfazer o que ele fez.

Ctrl + I

Instruções · Ferramentas · Modelo

Agent · Ask · Plan · Debug

Checkpoints

Fila de mensagens

Run Modes

Formação Claude Code 2026 · Baseado em cursor.com (Docs: Agent Overview, Prompting agents, Run Modes, Agent Review, Subagents; Help: Agent mode, Side chats) consultados em outubro de 2026 (Cursor 3.22) · Telas: reprodução ilustrativa

O que é *o Agent*

Você descreve a tarefa em linguagem natural e o Agent decide quais arquivos ler, que mudanças fazer e como verificar o resultado. As edições aparecem na tela enquanto ele trabalha.

COMO ABRIR

Painel lateral com `Ctrl` + `I` (Windows e Linux) ou `Cmd` + `I` (macOS). Também dá para rodar agentes pelo navegador em cursor.com/agents, com os Cloud Agents.

O QUE ELE CONSEGUE FAZER

- ✓ Criar uma funcionalidade do zero
- ✓ Refatorar código existente
- ✓ Corrigir bugs a partir de um erro ou sintoma
- ✓ Escrever testes e rodá-los
- ✓ Executar comandos no terminal e ler a saída

Para um trabalho maior, como uma migração, o Cursor tem os **Projects**: um agente coordenador planeja e distribui partes do trabalho para outros agentes.

Três peças *fazem um agente*

1 · INSTRUÇÕES

O **system prompt** do Cursor e as suas **rules** (do projeto, do usuário e do time). Dizem como o agente deve se comportar.

2 · FERRAMENTAS

Editar arquivos, buscar no código, rodar o terminal, usar o navegador e outras. São as "mãos" do agente.

3 · MODELO

O modelo que você escolhe para a tarefa (ou **Auto**). É quem decide o próximo passo.

PEDIDO

Você descreve a tarefa



DECIDE

O modelo escolhe a próxima ferramenta



EXECUTA

O Cursor roda a ferramenta



OBSERVA

O resultado volta para o modelo



REPETE

Até concluir e verificar

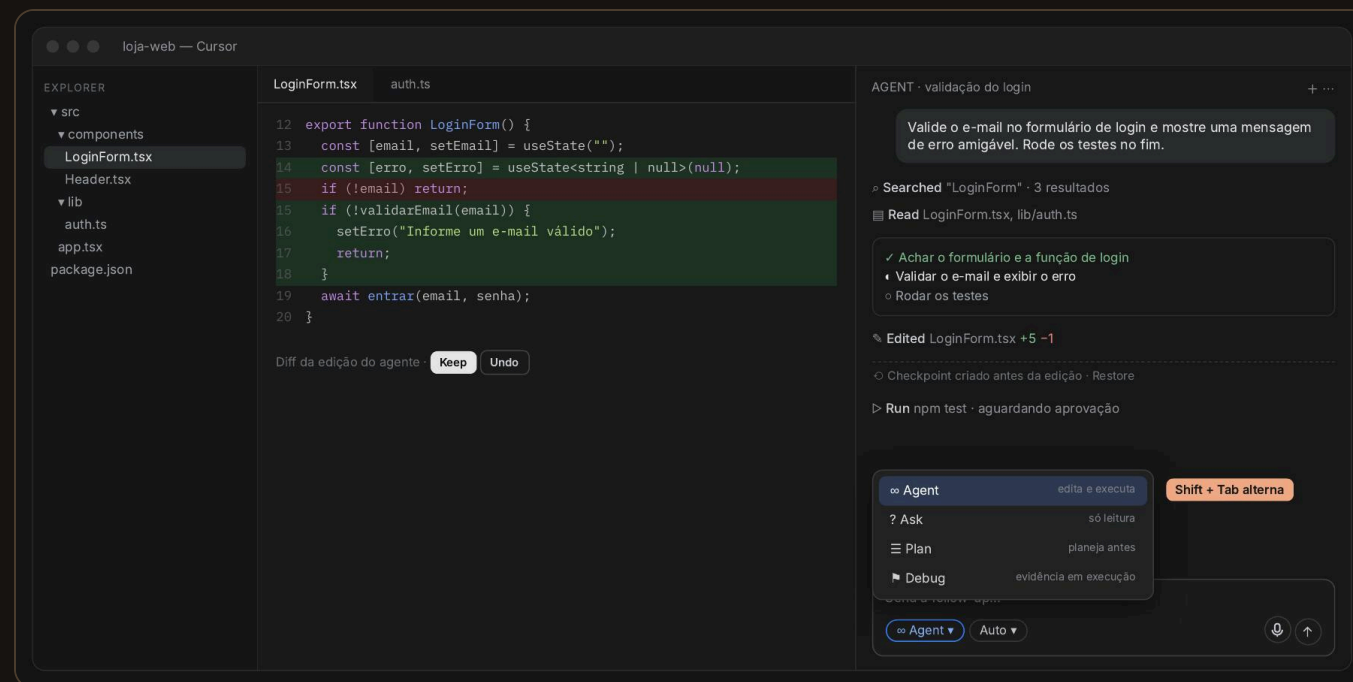
O Cursor ajusta instruções e ferramentas para **cada modelo** suportado. Quando sai um modelo novo, você só escolhe; a adaptação fica por conta do Cursor. **Não há limite** de chamadas de ferramenta numa tarefa.

A caixa de ferramentas *do Agent*

FERRAMENTA	O QUE FAZ	FERRAMENTA	O QUE FAZ
Buscar arquivos e pastas	Acha por nome, lê a estrutura de pastas, procura palavras e padrões	Rodar comandos	Executa no terminal e acompanha a saída (usa o primeiro perfil de terminal disponível)
Web	Monta buscas e pesquisa na internet	Navegador	Abre páginas, clica, tira prints e testa a aplicação (aula 09)
Buscar rules	Carrega rules pelo tipo e pela descrição	Gerar imagens	Cria imagens a partir de texto; salva em <code>assets/</code>
Ler arquivos	Lê o conteúdo; também lê imagens (.png, .jpg, .gif, .webp, .svg) para modelos com visão	Fazer perguntas	Pergunta algo no meio da tarefa e continua trabalhando enquanto espera a resposta
Editar arquivos	Sugere edições e aplica sozinho	MCP	Ferramentas extras de servidores conectados (aulas 05 a 09)

Para escolher o terminal que o agente usa: paleta de comandos (`Ctrl` + `Shift` + `P`) > **Terminal: Select Default Profile.**

O Agent *trabalhando*



Diff no editor, linha do tempo do agente e seletor de modos · reprodução ilustrativa

Quatro modos, *quatro jeitos de trabalhar*

MODO	MELHOR PARA	EDITA ARQUIVOS?
Agent	Criar funcionalidades, refatorar, corrigir bugs: a maioria das tarefas	Sim
Ask	Entender o código e a arquitetura (aula 03)	Não, só leitura
Plan	Funcionalidades complexas, quando você quer revisar a abordagem antes	Sim, depois que você aprova o plano
Debug	Bugs difíceis de reproduzir, que precisam de evidência em execução	Sim

Trocar de modo: `Shift` + `Tab` alterna entre eles, ou use o seletor no campo do chat.

Cada modo usa o **próprio contexto**: trocar de modo começa uma janela de contexto nova. As rules valem em todos os modos.

Dando uma tarefa *ao Agent*

- 1 Abra o painel com `Ctrl` + `I` e confira o modo (Agent) e o modelo.
- 2 Seja específico sobre o resultado: o que mudar, onde e como saber que deu certo.
- 3 Envie com Enter. O agente explora o projeto e começa a editar.
- 4 Acompanhe as edições no diff, enquanto acontecem.
- 5 Revise o que ficou e rejeite o que não quiser.

VAGO

Melhora o login.

ESPECÍFICO

Adicione um formulário de login na página inicial com campos de e-mail e senha. Valide o e-mail antes de enviar e mostre uma mensagem de erro amigável. Rode os testes no fim.

Se souber quais arquivos importam, cite com `@` (por exemplo, `@LoginForm.tsx`). Se não souber, deixe o agente buscar.

Revisar e *desfazer*

DIFF

As edições são aplicadas enquanto o agente trabalha. No **diff**, você vê cada mudança e pode rejeitar o que não quiser.

CHECKPOINTS

Antes de mudanças importantes, o agente salva um **checkpoint** dos arquivos. Clique num checkpoint da linha do tempo para ver como estavam e restaure tudo àquele ponto. Ou passe o mouse numa mensagem anterior e use **Restore Checkpoint**.

- ✓ Restaurar volta **só os arquivos**; as mensagens continuam na conversa
- ✓ Para encurtar a conversa, faça um **fork** a partir de uma mensagem anterior
- ✓ Checkpoints ficam **na sua máquina**, separados do Git

Checkpoint serve para desfazer o que o agente fez. Para histórico permanente, use **Git**: faça commits nos pontos bons.

O botão **Stop** interrompe o agente no meio da tarefa, para você corrigir o rumo ou tentar outra abordagem.

Falar com o agente enquanto ele trabalha

VOCÊ QUER	COMO	O QUE ACONTECE
Deixar o próximo pedido na fila	Digite e tecla <code>Enter</code>	Espera a tarefa atual acabar; dá para arrastar e reordenar a fila
Mandar já, sem esperar	<code>Ctrl</code> + <code>Enter</code> (<code>Cmd</code> + <code>Enter</code> no macOS)	Entra junto com os resultados das ferramentas e é processado na hora
Corrigir o rumo sem interromper	<code>Send now</code> ou <code>Enter</code> duas vezes (Agents Window e cursor.com/agents)	Chega na próxima chamada de ferramenta, sem cortar o que está em andamento
Tirar uma dúvida paralela	<code>/side</code> ou <code>/btw</code>	Abre um side chat que não enche a conversa principal
Dar um objetivo de longo prazo	<code>/goal</code> + objetivo	O agente trabalha até cumprir (recurso em liberação gradual)

Depois de fechada, a conversa ainda serve: o agente pode **buscar em conversas antigas** quando precisar de algo que vocês já discutiram.

Quem aprova *os comandos*

RUN MODE	COMO FUNCIONA	QUANDO USAR
Auto-review (padrão)	O que está na allowlist roda direto; comandos de terminal rodam na sandbox quando possível; o resto passa por um classificador de segurança	Menos interrupções, com revisão antes do que é mais arriscado
Allowlist	Só o que está na sua lista roda sem aprovação	Comportamento previsível, com poucas ações confiáveis
Run Everything	Tudo roda sozinho	Só se você aceita o risco (máquina ou pasta descartável)

Onde ajustar: Cursor Settings > **Agents** > **Approvals & Execution**. As ferramentas MCP seguem os mesmos Run Modes.

A documentação avisa: o Auto-review **não é uma barreira de segurança**. Continue lendo o que o agente pede para executar.

Subagentes e *revisão de código*

SUBAGENTES

O Agent pode criar **subagentes** para pesquisa, comandos ou navegador, em paralelo. Cada um tem a própria janela de contexto e devolve só o resultado. Os embutidos são **Explore**, **Bash** e **Browser**; você pode criar os seus com arquivos markdown em `.cursor/agents/`.

Use um subagente para achar todos os lugares onde validamos entrada do usuário.

AGENT REVIEW

Uma revisão de código dedicada às suas mudanças locais: digite `/agent-review`, rode pela aba **Source Control** (compara tudo com a branch principal) ou deixe automática após cada commit. Lê regras de arquivos `BUGBOT.md`.

PROFUNDIDADE	VELOCIDADE	CUSTO	MELHOR PARA
Quick	Rápida	Baixo	Mudanças pequenas, checagem rápida
Deep	Lenta	Alto	Lógica complexa, segurança, refatorações grandes

Sua primeira *tarefa com o Agent*

- 1 Crie uma pasta vazia, abra no Cursor e faça um `git init` (para ter um ponto de volta).
- 2 Peça ao Agent: *"Crie uma página HTML com um contador de cliques e um botão para zerar"*.
- 3 Enquanto ele trabalha, coloque na fila: *"Depois, deixe o botão maior e centralizado"*.
- 4 Rejeite uma das edições no diff e veja o arquivo voltar.
- 5 Restaure o primeiro checkpoint e compare com o estado final.

O objetivo não é o contador, e sim sentir o ciclo: **pedir, acompanhar, corrigir e desfazer**.

Troque para **Ask** com `Shift` + `Tab` e pergunte *"como esse contador funciona?"*: repare que, nesse modo, nada é editado.

Teste o que *ficou*

1. Quais são as três peças de um agente no Cursor?

A) Editor, terminal e Git

B) Instruções, ferramentas e modelo

C) Chat, Tab e Ctrl+K

D) Rules, skills e hooks

B. O modelo decide, as ferramentas agem e as instruções orientam.

2. O que acontece ao restaurar um checkpoint?

A) Apaga a conversa

B) Os arquivos voltam; as mensagens ficam

C) Faz um commit no Git

D) Troca o modelo

B. Para encurtar a conversa, use um fork.

3. Qual modo responde sem editar nada?

A) Agent

B) Plan

C) Ask

D) Debug

C. Ask é só leitura.

4. Com o agente trabalhando, você tecla Enter numa mensagem nova. O que acontece?

A) A tarefa é cancelada

B) A mensagem vai para a fila

C) A mensagem é descartada

D) Abre um side chat

B. Para mandar já, use Ctrl + Enter.

Você pede, *o agente executa*

O Agent junta **instruções**, **ferramentas** e um **modelo** para concluir tarefas: busca, edita, roda comandos e verifica. Escolha o modo certo (Agent, Ask, Plan ou Debug), acompanhe pelo diff, corrija o rumo com a fila e o Ctrl + Enter, e desfaça com checkpoints, sem esquecer do Git e dos Run Modes.

Ctrl + I

Shift + Tab

Restore Checkpoint

Fila · Ctrl + Enter

/agent-review

Próxima aula → 02 · Interagindo com o Agente via Comando de Voz

Fontes: cursor.com/docs (Agent Overview, Prompting agents, Run Modes, Agent Review, Subagents); cursor.com/help (Agent mode, Side chats), consultados em outubro de 2026. A tela é uma reprodução ilustrativa.